Wazuh SIEM/XDR

Improvement Plan

OFIR LTD — Osijek, Croatia

Project: Production SIEM Pipeline
MikroTik → Graylog → Wazuh → OpenSearch

Author: Mohamed Amine Namouchi
Date: June 2026

Current Infrastructure Status

- ✓ 6 MikroTik routers sending CEF logs over TLS to Graylog
- ✓ Graylog → rsyslog → file → Wazuh Agent (AES-256)
- ✓ Custom PCRE2 decoders for MikroTik log parsing
- ✓ Brute-force detection rules (T1110)
- ✓ Active response scripts (Mattermost + email alerts)
- ✓ OpenSearch dashboards (global SOC + per-router views)

This document serves as technical annex for the internship report
and as a reference roadmap for future SOC improvements at OFIR LTD.

Contents

Infrastructure & Architecture

Objective

Stabilize and document the Wazuh deployment foundation to prevent silent failures and ensure reliable operation of all components.

- ☐ CRITICAL Fix Wazuh Manager IP to **static assignment** — prevent pipeline failures caused by DHCP-induced IP changes (previously caused a full outage when manager IP changed)
- ☐ HIGH Define a **high availability strategy** — what happens if the Wazuh server goes down? Document failover and recovery procedures
- ☐ HIGH Document the **exact log format** expected by Wazuh decoders (syslog converted by Graylog output plugin, not raw CEF) — prevent silent decoder breakage if Graylog config changes
- ☐ HIGH Create a **complete visual architecture diagram** of OFIR — all assets, data flows, monitored vs unmonitored components
- ☐ MEDIUM Implement **component health monitoring** — alert if Wazuh manager, indexer, or dashboard becomes unreachable
- ☐ MEDIUM Define a **configuration backup strategy** — automated backup of `/var/ossec/etc/` to enable rapid reconstruction after a server failure

Visibility & Log Collection

Objective

Eliminate blind spots by extending log collection to all critical infrastructure components, not only MikroTik routers.

- ☐ CRITICAL Deploy a **dedicated Wazuh agent on the Graylog server** (group: `graylog-server`) — collect auth logs, SSH events, sudo usage, service health, and application logs
- ☐ CRITICAL Collect **Wazuh server own system logs** — a SIEM that does not monitor itself is a critical blind spot
- ☐ HIGH Create a **log source inventory document** — for each source: owner, collection method, format, retention period, and detection value
- ☐ HIGH Collect **Graylog application logs** (not only system logs) — Graylog startup, stream errors, pipeline failures
- ☐ HIGH Evaluate **other network devices** for syslog forwarding — switches, other network gear at OFIR
- ☐ MEDIUM Define **per-source retention policy** — how long each log source is kept (MikroTik logs vs auth logs vs application logs may require different durations)
- ☐ MEDIUM Implement **agent health monitoring** — alert when an agent has been disconnected for more than X minutes

Detection Engineering

Objective

Enrich and extend detection capabilities with proper MITRE ATT&CK mapping, compliance tagging, and additional use cases covering attack patterns not yet detected.

- ☐ CRITICAL Add **MITRE ATT&CK mapping** to all existing rules — at minimum `<mitre><id>T1110</id></mitre>` on brute-force rules (Credential Access)
- ☐ CRITICAL Add **compliance tags** to existing rules — `gdpr_IV_35.7.d`, `nist_800-53_AU-6`, `cis_v8_8.2` in the `<group>` field
- ☐ HIGH Create **CDB Lists** for authorized admin IPs — prevent false positives on legitimate admin traffic to MikroTik routers
- ☐ HIGH Create a **password spraying detection rule** using `<same_user />` — detect distributed attacks targeting one username from multiple source IPs (T1110.003)
- ☐ HIGH Create an **auth success after brute-force rule** — alert when a login succeeds shortly after multiple failures from the same IP (potential successful compromise)
- ☐ HIGH **Test all existing rules** with `wazuh-logtest` and document sample logs for each decoder
- ☐ HIGH Create a **multi-source correlation rule** — brute-force on MikroTik + SSH connection on Graylog server from same IP within 1 hour = critical alert
- ☐ MEDIUM **Document all custom decoders** — extracted fields, regex explanation, sample log entry, and expected output
- ☐ MEDIUM Create and maintain **sample log files** for each decoder — used to validate after any modification
- ☐ MEDIUM Define **CVE remediation SLAs**: Critical (CVSS $\geq$ 9.0) $\rightarrow$ 48h, High (7.0–8.9) $\rightarrow$ 7 days, Medium (4.0–6.9) $\rightarrow$ 30 days, Low ($<$ 4.0) $\rightarrow$ next cycle
- ☐ MEDIUM **Audit Java, MongoDB/OpenSearch versions** on the Graylog server — frequently vulnerable components

Use Cases & Playbooks

Objective

Transform isolated detection rules into complete operational playbooks that guide analysts from alert triage to incident closure.

- ☐ CRITICAL Write a **complete playbook for the existing brute-force rule** — step-by-step: identify $\rightarrow$ validate $\rightarrow$ investigate $\rightarrow$ contain $\rightarrow$ close
- ☐ CRITICAL Create a **Log Clearing use case** — monitor `/var/ossec/logs/`, `/var/log/auth.log`, Graylog logs — trigger critical alert if deletion detected (T1070)
- ☐ HIGH Create a **Graylog server use case** — SSH brute-force, sudo abuse, unexpected service restart, suspicious connections (new agent group needed)
- ☐ HIGH Write a **complete SOC runbook** following NIST IR structure: Preparation $\rightarrow$ Detection & Analysis $\rightarrow$ Containment $\rightarrow$ Eradication $\rightarrow$ Recovery $\rightarrow$ Post-Incident Activity
- ☐ HIGH Create an **alert triage checklist** — standardized steps for every incoming alert regardless of type

- ☐ MEDIUM Transform the **MikroTik reboot detection rule** into a full use case with playbook
- ☐ MEDIUM Transform the **MikroTik config change detection rule** into a full use case with playbook
- ☐ MEDIUM Create a **resource monitoring use case** — CPU, RAM, disk spikes on Graylog and Wazuh servers
- ☐ MEDIUM Define **escalation levels** based on alert severity:
  Level 10 → Mattermost notification, Level 13+ → phone call, Level 15 → emergency response plan
- ☐ MEDIUM **Test the runbook** with a simulated incident — validate that all steps work as expected under realistic conditions
- ☐ MEDIUM **Document change windows** — define authorized maintenance periods to distinguish legitimate changes from suspicious ones

File Integrity Monitoring (FIM)

Objective

Detect unauthorized modifications to critical system files, security tool configurations, and log pipeline components before they cause undetected damage.

- ☐ CRITICAL Activate FIM on `/var/ossec/etc/` — highest priority: if `ossec.conf` is modified silently, the entire detection platform can be disabled
- ☐ CRITICAL Activate FIM on `/var/ossec/etc/decoders/` and `/var/ossec/etc/rules/` — modified decoders or rules can silently kill all detections
- ☐ CRITICAL Activate FIM on `/var/ossec/active-response/bin/` — if response scripts are tampered with, automated defenses may behave unpredictably
- ☐ HIGH Activate FIM on `/etc/graylog/server/server.conf` — if Graylog config is modified, the log pipeline can be redirected or disabled
- ☐ HIGH Activate FIM on `/etc/rsyslog.conf` and `/etc/rsyslog.d/` — if rsyslog config changes, MikroTik logs may stop arriving silently
- ☐ HIGH Activate FIM on `/etc/passwd`, `/etc/shadow`, `/etc/sudoers` — unauthorized user creation or privilege escalation
- ☐ HIGH Activate FIM on `/etc/ssh/sshd_config` — detect if SSH is reconfigured to allow root login or password authentication
- ☐ HIGH Activate FIM on `/etc/crontab` and `/etc/cron.d/` — detect persistence via scheduled tasks (T1053)
- ☐ HIGH Activate FIM on `/var/log/auth.log` — detect if auth logs are deleted or truncated (T1070)
- ☐ MEDIUM Configure FIM to capture **full metadata** — hash (MD5 + SHA256), file owner, permissions, modification time, and process that triggered the change
- ☐ MEDIUM Create a **FIM use case with playbook** — critical alert when monitored files change outside change windows
- ☐ MEDIUM **Document file owners** for each monitored path — who validates when a FIM alert fires on that path?

SCA & Vulnerability Detection

Objective

Identify vulnerable software versions and insecure configurations across all monitored servers before attackers can exploit them.

- ☐ CRITICAL **Activate Vulnerability Detection** on all Wazuh agents — inventory installed packages and match against CVE database
- ☐ CRITICAL **Activate SCA** with CIS Ubuntu 22.04 benchmark on both Graylog and Wazuh servers
- ☐ HIGH **Verify SSH configuration** on both servers: `PermitRootLogin no`, `PasswordAuthentication no`, `Protocol 2` only
- ☐ HIGH **Audit Java version** on the Graylog server — Java is a frequent target; check against current CVEs and verify Graylog compatibility before patching
- ☐ HIGH **Audit MongoDB / OpenSearch version** used by Graylog — check for known CVEs and ensure version is supported
- ☐ HIGH Define and enforce **CVE remediation SLAs** — assign ownership for patching (who patches what at OFIR?)
- ☐ MEDIUM Establish a **patch verification process** — after patching, re-run vulnerability scan to confirm CVE closure

Active Response

Objective

Make existing active response scripts safer, more auditable, and more effective by adding whitelisting, logging, and additional response actions.

- ☐ CRITICAL Create a **CDB List of critical IPs to whitelist** before any automated action fires — include Graylog server IP, Wazuh server IP, admin workstations, and authorized scanners
- ☐ CRITICAL **Verify Graylog server IP is whitelisted** first — if active response blocks the Graylog IP, the entire log pipeline collapses silently
- ☐ HIGH **Verify timeout is set** in all scripts — `INPUT=$(timeout 3 cat)` prevents stdin deadlock (already partially implemented — verify all scripts)
- ☐ HIGH **Improve action logging** in all scripts — each automated action must log: timestamp, source IP, triggering rule ID, action taken, and result
- ☐ HIGH **Add iptables-level IP blocking script** — complement Mattermost/email notifications with actual network-level enforcement
- ☐ HIGH **Test all scripts with intentional false positives** — validate whitelist behavior, timeout, logging, and rollback in a controlled scenario
- ☐ HIGH Write a **manual rollback procedure** — documented steps to unblock an IP immediately if active response fires incorrectly
- ☐ MEDIUM **Monitor active response scripts with FIM** — if a script is modified, alert immediately (defense evasion risk)
- ☐ MEDIUM Test **agent behavior when manager is unreachable** — does the agent queue events? Does active response still fire locally?

Integrations

Objective

Connect Wazuh with complementary security tools to enable faster triage, richer context, and more complete incident tracking.

- ☐ HIGH **Activate VirusTotal integration** — automatic hash reputation lookup when FIM detects a new file; requires only a free API key in `ossec.conf`
- ☐ HIGH **Enrich Mattermost notifications** with:
  - MITRE ATT&CK tactic and technique ID
  - IP geolocation (country + ASN/ISP)
  - VirusTotal reputation score
  - Occurrence counter ("3rd alert from this IP in 24h")
  - Active response status ("IP blocked for 10 minutes")
  - Direct link to the alert in Wazuh dashboard
  - Link to the corresponding playbook
- ☐ HIGH **Deploy TheHive** (open-source) — replace informal Mattermost tracking with structured incident management: case creation, analyst assignment, evidence storage, MTTR measurement
- ☐ MEDIUM **Evaluate Shuffle SOAR** (open-source) — automate the full workflow: Wazuh alert → VirusTotal lookup → TheHive case creation → Mattermost notification
- ☐ MEDIUM **Document the SOC integration workflow** — even the current simple version: alert → Mattermost → manual investigation → resolution

Compliance & SOC Operations

Objective

Establish continuous compliance monitoring and measurable SOC operations metrics to demonstrate security maturity at OFIR.

- ☐ CRITICAL **Define log retention policy** — how long are Wazuh alerts, MikroTik logs, and auth logs retained? Must align with GDPR requirements (minimum duration for incident investigation)
- ☐ HIGH **Add compliance tags to existing rules** — GDPR (`gdpr_IV_35.7.d`), NIST, CIS Controls tags enable built-in compliance dashboard views
- ☐ HIGH **Activate compliance dashboards** in Wazuh — GDPR and CIS Controls views as first priority for OFIR
- ☐ HIGH **Define GDPR breach notification process** — if a data breach is detected, who notifies whom within the mandatory 72-hour window?
- ☐ HIGH **Start an incident register** — a simple spreadsheet listing all security incidents: date, type, severity, action taken, resolution, and lessons learned
- ☐ MEDIUM **Create a monthly security report** for Domagoj — alert counts by severity, top attacking IPs, FIM events, SCA compliance score, open CVEs
- ☐ MEDIUM **Document security controls in place** — written evidence that monitoring is continuous (required for any future audit)

- ☐ MEDIUM **Measure SOC KPIs** — even manually at first:
  - MTTD (Mean Time to Detect)
  - MTTA (Mean Time to Acknowledge)
  - MTTR (Mean Time to Respond/Recover)
  - False Positive Rate
  - Alert volume by rule and severity
- ☐ MEDIUM **Retrospectively document past incidents** — brute-force events already observed and handled
- ☐ MEDIUM **Define a Lessons Learned process** — after each incident: short debrief with Domagoj, rule tuning if needed, playbook update

Document Information

Author	Mohamed Amine Namouchi
Company	OFIR LTD, Osijek, Croatia
School	Polytech Dijon — Engineering in Security & Network Quality
Supervisor	Domagoj Muhar
Version	1.0
Date	June 2026
Based on	Practical internship experience at OFIR LTD